



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Chong Hernández Samuel

N° de Cuenta: 319239504

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 22 de febrero del 2025

CALIFICACIÓN: _____

Reporte de práctica.

Ejercicios realizados

1. Dibujar las iniciales de su nombre, cada una con un color diferente.

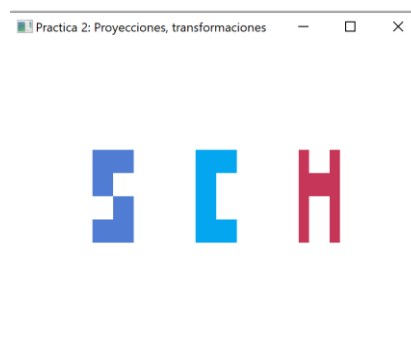
Para resolver el primer ejercicio de la práctica únicamente se copiaron los vértices de la práctica pasada, hablando de aquellos que definen las letras. Con la adición de añadir los tres parámetros para definir su color, haciendo que cada una se muestre de diferente color.

```
GLfloat vertices_letra_S[] {  
    -0.4f,  0.1f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.2f,  0.1f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.2f,  0.2f, 0.5f,  0.312f, 0.491f, 0.828f, //estos vértices hacen la parte superior de la S  
    -0.4f,  0.1f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.2f,  0.2f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.4f,  0.2f, 0.5f,  0.312f, 0.491f, 0.828f,  
  
    -0.4f,  0.0f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.3f,  0.0f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.3f,  0.1f, 0.5f,  0.312f, 0.491f, 0.828f, //esos vértices hacen la parte izquierda central de la  
    -0.4f,  0.0f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.3f,  0.1f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.4f,  0.1f, 0.5f,  0.312f, 0.491f, 0.828f,  
  
    -0.3f, -0.1f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.2f, -0.1f, 0.5f,  0.312f, 0.491f, 0.828f, //estos vértices hacen la parte derecha central de  
    -0.2f,  0.0f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.3f, -0.1f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.2f,  0.0f, 0.5f,  0.312f, 0.491f, 0.828f,  
    -0.3f,  0.0f, 0.5f,  0.312f, 0.491f, 0.828f,  
}
```

A continuación se muestra la forma en la que se dibujó cada letra, utilizando las funciones usadas previamente para trasladar y escalar cada letra de manera que queden lo mas centradas posibles.

```
////Letra S  
shaderList[1].useShader();  
uniformModel = shaderList[1].getModelLocation();  
uniformProjection = shaderList[1].getProjectLocation();  
  
////Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geométr  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-0.2f, 0.0f, -2.0f));  
model = glm::scale(model, glm::vec3(1.0f, 1.5f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA y se envi  
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));  
meshColorList[7] -> RenderMeshColor();  
  
////Letra C  
shaderList[1].useShader();  
uniformModel = shaderList[1].getModelLocation();  
uniformProjection = shaderList[1].getProjectLocation();  
  
////Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geométr  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));  
model = glm::scale(model, glm::vec3(1.0f, 1.5f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA y se envi  
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));  
meshColorList[8] -> RenderMeshColor();
```

Ejecución:



Podemos observar que las letras hechas la práctica pasada ahora se muestran con un color diferente cada una.

2. *Generar el dibujo de la casa de la clase, pero en lugar de instanciar triangulos y cuadrados será instanciando piramides y cubos, para esto se requiere crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp.*

En este ejercicio se declaró en la parte de `createShaders()` a cada shader por color, haciendo referencia al `.frag` que próximamente se crearía para cada uno de los solicitados.

```
//1
Shader* shaderRojo = new Shader();
shaderRojo->CreateFromFiles(vShader, "rojo.frag");
shaderList.push_back(*shaderRojo);

//2
Shader* shaderVerde = new Shader();
shaderVerde->CreateFromFiles(vShader, "verde.frag");
shaderList.push_back(*shaderVerde);

//3
Shader* shaderAzul = new Shader();
shaderAzul->CreateFromFiles(vShader, "azul.frag");
shaderList.push_back(*shaderAzul);

//4
Shader* shaderCafe = new Shader();
shaderCafe->CreateFromFiles(vShader, "cafe.frag");
shaderList.push_back(*shaderCafe);

//5
Shader* shaderVerdeOsc = new Shader();
shaderVerdeOsc->CreateFromFiles(vShader, "verdeOscuro.frag");
shaderList.push_back(*shaderVerdeOsc);
```

Se creó un `.frag` por cada color.

- ☐ azul.frag
- ☐ cafe.frag
- ☐ rojo.frag
- ☐ shader.frag
- ☐ shader.vert
- ☐ shadercolor.frag
- ☐ shadercolor.vert
- ☐ verde.frag
- ☐ verdeOscuro.frag

De esta manera, solo fue cuestión de cambiar en las figuras previamente definidas en el ejercicio de sesión, al índice en la lista de *shaders* poniendo el correspondiente, tanto para la definición del color así como de pasar de cuadrado a cubo y triángulo a pirámide.

```
////triangulo azul
shaderList[3].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

////Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transf
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.45f, -2.0f));
model = glm::scale(model, glm::vec3(1.2f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRA
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]->RenderMesh();

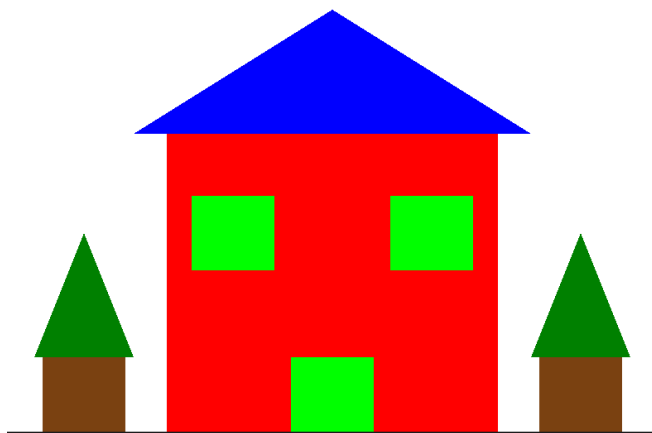
////cuadrado verde inferior
shaderList[2].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

////Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transf
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -0.85f, -1.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.3f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRA
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
```

Ejecución:

Podemos observar que se obtiene un dibujo parecido al obtenido en el ejercicio de sesión, cumpliendo así con el ejercicio de la práctica.

Practica 2: Proyecciones, transformaciones — □ ×



Problemas presentados

Dado que el mayor porcentaje del código consistió en copiar código previamente utilizado, el único problema que se presentó fue en la definición o creación de los *shaders* para el segundo ejercicio, específicamente en la parte donde se hace referencia a los *.frag* a los que apuntan.

Analizando el código, se entendió la forma en que funcionan y se pudieron definir de forma correcta.

Conclusión

Concluyo que la práctica genera un conocimiento sólido en cuanto al manejo de colores y formas por medio de los *shaders*, comprendiendo su forma de definirlos y llamarlos. Esto pasando de un 2D, donde se observó una forma de asignar un color diferente a cada figura, a un 3D donde se hace lo mismo pero para cuerpos geométricos.

En cuanto a la complejidad de la práctica la considero media, más que nada porque no se requirió del uso de lógica computacional o algo por el estilo, sino por saber definir cada shader.

Para finalizar considero que fue una buena práctica, los ejercicios permiten un mayor adentramiento en el funcionamiento de OpenGL que nos ayuda a saber utilizarlo y manejarlo según nuestros intereses como lo será el futuro proyecto final.

Bibliografía

- Swiftless. (n.d.). OpenGL scaling. [Sitio web]. Swiftless.
https://www.swiftless.com/tutorials/opengl/scaling.html#google_vignette